

Saturday, July 2, 2022

10:33 AM

```
FOR DELETE COMMAND
INSERT INTO EMP SELECT * FROM EMP
AS OF TIMESTAMP(SYSDATE-INTERVAL'5'
MINUTE);
```

For Correct set pages 250 lines 250

How to set page:

command is=

```
set pagesize 20;
set linesize 120;
```

Notes Link:

<https://bit.ly/oracle730pm>

```
select * from emp where sal=(select max(sal) from emp where sal<(select max(sal) from emp));
```

Oracle DataBase 19c installation process

<https://bit.ly/oracleinstall>

index is a db object that makes data retrieval faster.

indexe created on column and that column is called index key. Default index is BTREE

BTREE INDEX.

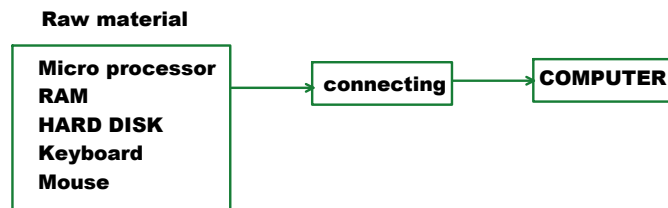
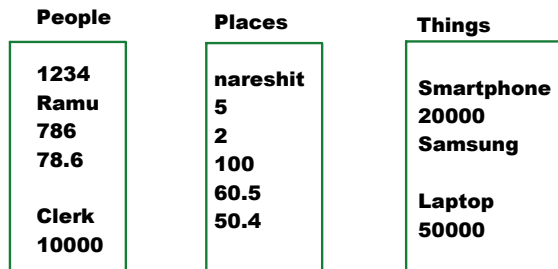
BITMAP INDEX.

How to create index. create index i1 on emp(sal)

Data
Information
Database
DBMS
RDBMS
Metadata

Data:

- Data is a raw collection of facts about people, places, things ...etc.

**Data:**

- Data is a raw collection of facts about people or places or things..etc.
- It is unprocessed one.
- It is not in meaningful form.
- It cannot be used for data analysis.

Information:

- If we arrange the data in meaningful form then it is called "Information".
- It is processed one.
- It is in meaningful form.
- It can be used for Data Analysis.

Types of Data:**2 types:**

- Structured Data
- Unstructured Data

Structured data:

It is made up of with letters, digits & special symbols.

Ex: Ramu 1234 H.NO.1-2-123

Unstructured Data:

Images, audios, videos, documents, animations are comes under unstructured data.

employee

empno	ename	job	sal	epphoto
1234	Kiran	CLERK	5000	10110001001001010

Database:**Bank DB**

Customers
Transactions
Branches
Staff

College DB

Student
Marks
Fee
Staff
Library

Amazon DB

Products
Customers
Orders

- Database means, complete details of an organization stored in computer in specific manner.
- Database is a collection of interrelated data in an organized form.

College DB

STUDENT		
sid	sname	scity
1001	Ravi	Hyd

MARKS			
sid	math	phy	che
1001	60	80	70

FEE			
sid	total_fee	paid_fee	balance
1001	15000	10000	5000

Interrelated data

STUDENT			Table / Relation / Entity
sid	sname	scity	
1001	Ravi	Hyd	row / tuple / entity instance / record
1002	Kiran	Delhi	
1003	Ramu	Mumbai	Column / Field / Attribute / Property

Table:

is a collection of rows & columns

Field:

holds individual values

Record:

is a collection of field values

Database contains records & fields

DBMS:

- DBMS => DataBase Management Software / System
- It is a software that is used to create & maintain the database.
- Using this software, we can store, manipulate & retrieve the data of database.
 - manipulate => INSERT / UPDATE [modify] / DELETE
 - retrieve => opening existing data

RDBMS:

- RDBMS => Relational DataBase Management Software / System.
- Relation => Table
- RDBMS is a software that is used to create & maintain the database in the form of tables [relations].

Ex:

ORACLE SQL SERVER My SQL DB2 Postgre SQL

Laptop RDBMS

apple	oracle
dell	sql server
hp	my sql
asus	db2

Metadata:

- It is the data about the data.
 - It describes about the data.
 - Ex:
 - Table name [STUDENT_INFO]
 - column name [sid,sname,scity]
 - Data type
 - Field size => [sid => number(4)]
- Handwritten notes: "Data Type" with an arrow pointing to "number(4)" and "Field Size" with an arrow pointing to "(4)".*

STUDENT_INFO

sid	sname	scity
1234	Ramu	Hyd
1235	Kiran	Mumbai

int
float
char

sid Number(4) => -9999 to 9999

1234 *Handwritten notes: "Data Type" with an arrow pointing to "Number(4)" and "Field Size" with an arrow pointing to "(4)".*
1235
1236

sname Varchar2(10)

sname	Varchar2(10)

Vijay	
Arun	

Data Type field size

ORACLE

- ORACLE is a Relational DataBase Management Software [RDBMS]
- Relation => Table
- ORACLE is used to create & maintain database in the form of tables.
- This software allows us to store, manipulate & retrieve the data of database.
- manipulate => INSERT / UPDATE / DELETE
 - Ex:
 - emp joined => INSERT
 - emp got promotion => UPDATE
 - emp resigned => DELETE
- RETRIEVE => opening existing data
 - Ex: check balance online shopping
- ORACLE software is product of ORACLE company.
- ORACLE software 2nd version introduced in 1979.
- ORACLE 21C is latest version => 2021.

ORACLE => RDBMS

RDBMS => Concept => introduced in 1976 => E.F.Codd

Before 1960s => data manually in books

1960s => FMS [File Management System]

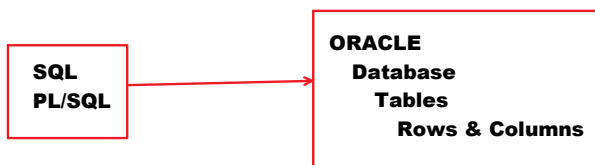
1970s => Hierarchical & Network DBMSs

ORACLE company owner => LARRY ELLISON

1977 => Software Development Laboratories

1979 => Relational Software Inc.

1983 => ORACLE corp.



To deal with ORACLE database, we use 2 languages:

- SQL
- PL/SQL

SQL:

- SQL => Structured Query Language
- SQL is a Query Language that is used to write the queries.
- Query => is a request that is sent to DataBase Server.
- Queries are written to communicate with ORACLE database.
- SQL is a Non-Procedural Language. We will not write any set of statements.
- SQL is a 4GL [4th Generation language]. In 4GLs, we much focus on what to do rather than how to do. These 4GLs provides readymade commands, functions to make our tasks easier.
- SQL provides operators to perform operations.
- SQL provides built-in functions.

C /Java / Python

Programming Languages
programs

Function [in C]:
set of statements

Method [in Java]:
set of statements

Procedure [PL/SQL]:
set of statements

ORACLE SQL provides 5 sub Languages. They are:

DDL Data Definition Language	CREATE ALTER TRUNCATE
---------------------------------	-----------------------------

	DROP RENAME FLASHBACK [Oracle 10g] PURGE [Oracle 10g]
DRL / DQL Data Retrieval Language Data Query Language	SELECT
DML Data Manipulation Language	INSERT UPDATE DELETE INSERT ALL [Oracle 9i] MERGE [Oracle 9i]
DCL / ACL Data Control Language Accessing Control Language	GRANT REVOKE
TCL Transaction Control Language	COMMIT ROLLBACK SAVEPOINT

DDL :

- DDL stands for Data Definition Language.
- It deals with metadata.

SQL provides following DDL commands:

- Create
- Alter
- Drop
- Truncate
- Rename
- Flashback [Oracle 10g]
- Purge [Oracle 10g]

CREATE:

- It is used to create the DataBase Objects like tables, views, indexes ...etc.

Syntax to CREATE the table:

```
CREATE TABLE <table_name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  .
.]
);
```

DB Objects

- Table
- View
- Index
- Materialized view
- Sequence
- Synonym
- Procedure
- Function
- Package

Data Types in SQL:

- Data Type tells which type of data a column has to accept.

employee

empno	ename	job	sal	deptno
-------	-------	-----	-----	--------

ORACLE SQL provides following data types:

Number Related	Number(p) Int Integer	<i>Integer</i> <i>Rel Int</i>
----------------	-----------------------------	----------------------------------

Number Related ✖	Number(p) ✓ Int Integer	Integer Related
	✖ ✖ ✖ Number(p,s) ✓ Float binary_float binary_double Real	
Character Related	Char(n) Varchar2(n) LONG CLOB nChar(n) nVarchar2(n) nCLOB n => national	
Date & Time Related	Date Timestamp (Oracle 9i)	
Binary Related	BFILE BLOB	

Number Related Data Types:

Number(p):

- It is used to hold integers.
- Integer means, number without decimal places.
- p => precision => max no of digits
- precision valid range => 1 to 38

Ex:

max marks: 100

0 to 100

maths_marks Number(3)

100 => 3 digits

78

100

999

-999 TO 999

1000 --ERROR

1000 --ERROR

123456 -- ERROR

Addhar_num Number(12)

Mobile_num Number(10)

cust_id Number(6)

123456

123457

123458

age Number(2)

18

23

21

25

Number(38) = INT = INTEGER

```
CREATE TABLE t3
(
  f1 number(38),
  f2 int,
  f3 integer
);
```

Number(p,s):

- **p => precision => max number of digits**
- **s => scale => max no of decimal places**
- **It is used to hold floating point values.**

Ex:

3 subject

avrg **Number(5,2)**

 -999.99 TO 999.99
 78.23
 83.45
 100.00
 999.99
 1000 --ERROR

math => 100
 phy => 100
 che => 100

average => 100.00

5,2

sal **Number(8,2)** **max sal => 100000.00**

999999.99

height **Number(2,1)**

5.3
 5.2 -9.9 TO 9.9
 5.8
 5.9
 6.0

Character Related data types:

Char(n):

- **n => max no of chars**
- **used to hold a group of chars [string values].**
- **Fixed Length Data Type.**
- **max size: 2000 bytes.**
- **default size: 1**

Varchar2(n):

- **n => max no of chars**
- **used to hold a group of chars [string values].**
- **Variable Length data type.**

- max size: 4000 bytes.
- default size: no default size

f1 Char(10)	f2 Varchar2(10)
ramu6spaces	ramu
ramesh4spaces	ramesh
sai7spaces	sai

Handwritten notes:

- For f1 Char(10): ramu (6 spaces), ramesh (4 spaces), sai (7 spaces). Total length is 10 for each.
- For f2 Varchar2(10): ramu (4), ramesh (6), sai (3). Total length is 10 for each.

create table t5(f1 char); --default size 1

create table t6(f1 varchar2); --no default size
--ERROR

Vehicle_num	Char(10)	ename	Varchar2(10)
TS09AA1234		ramu	
		kiran	
		naresh	
		sai	

PAN_CARD_NUM	Char(10)	pname	varchar2(15)
ABCDE 12345		laptop	
		keyboard	
		hard disk	

Gender	Char(1)
M	
F	

StateCOde	Char(2)
AP	
TS	
MH	
WB	

CountryCode	Char(3)
IND	
AUS	
PAK	
USA	

LONG:

- used to hold string values.
- max size: 2 GB.
- It is used to hold large amounts of chars.
- In one table, we create one column as LONG type.
- We cannot use built-in functions on LONG type.

Ex:

Customer_Feedback
Experience summary
Cust_Complaints
Product_features

CLOB:

- CLOB => Character Large Object
- Max size: 4 GB
- it is used to hold string values.
- it is used to hold large amounts of chars.

Ex:

Customer_Feedback
Experience summary
Cust_Complaints
Product_features

Char(n) Varchar2(n) LONG CLOB	ASCII CODE char data types Single Byte Char data types supports to ENGLISH language chars only.
nChar(n) nVarchar2(n) nCLOB	UNI CODE char data types Multi Byte char data types supports to ENGLISH + other language chars also

n => national character set data types

In C =>

char x; // 1 Byte = 8 bits => ASCII
0 to 255
255 => 1111 1111 => 8 bits
256 => 9 bits
English chars only

In Java:

char x; // 2 bytes => UNI code
0 to 65535

65535 => 1111 1111 1111 1111 => 16 bits
65536 => 17 bits

Java => English + other lang chars

Date & Time related data types:**Date:**

- It is used to hold date values.
- It can hold date, month, year, hours, minutes & seconds.
- Default date format: dd-mon-rr
- Ex: 23-DEC-19
- default time: 12:00:00 AM [mid night]
- Fixed length data type.
- size: 7 bytes

Ex:

DOBIRTH
DOJoining
DORetirement
ord_date
del_date

yy

2019
2078

RR

	2022	2070
00-49	19 => 2019 25 => 2025	19 => 2119 25 => 2125
50-99	78 => 1978 85 => 1985	78 => 2078 85 => 2085

21st century

2000 to 2049
2050 to 2099

Timestamp:

- used to hold date & time values.
- It can hold date, month, year, hours, minutes, seconds & fractional seconds.
- Fixed length data type
- size: 11 bytes
- It is extension of date data type.

Ex

date_of_joining

.....

bill_date_time

.....

Date	Timestamp
7 bytes	11 bytes
It cannot hold fractional seconds	It can hold fractional seconds
it cannot display time value by default. to see it, we use "to_Char()" function	it displays time values by default.

Binary related data types:

BFILE:

- used to maintain multimedia objects like images, audios, videos, animations, documents.
- It maintains path of multimedia object.
- External Large Object.
- It is not secured one

Database

emp		
empno	ename	ephoto {BFILE}
1234	ramu	d:\employees\ramu.jpg

d:
employees folder
ramu.jpg



BLOB:

- used to maintain multimedia objects like images, audios, videos, animations, documents.
- multimedia object will be stored inside of table.
- Internal Large Object.

emp		
empno	ename	ephoto {BLOB}
1234	ramu	1253AB8745CDF123EF

d:
employees folder
ramu.jpg



Database

It is secured one.

Log In as DBA:

username: system

password: nareshit

[at the time of installation u have given this password]

CREATING USER:

Syntax:

```
CREATE USER <user_name>
IDENTIFIED BY <password>
DEFAULT TABLESPACE <tablespace_name>
QUOTA <size> ON <tablespace_name>;
```

TABLESPACE	is a file. in this oracle data will be stored
------------	---

There are 2 types of users:

- Local User
- Common User

Creating Common user:

Common user name must be prefixed with c##.

Ex:

```
CREATE USER c##batch4pm
IDENTIFIED BY nareshit
DEFAULT TABLESPACE users
QUOTA unlimited ON users;
```

```
GRANT connect,resource TO c##batch4pm;
```

Examples on creating tables:

Ex-1:

STUDENT

sid	sname	m1
-----	-------	----

Syntax of CREATE command:

```
CREATE TABLE <table_name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>],
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  .
.]
);
```

CREATE TABLE student

(

STUDENT

CREATE TABLE student

```
(
  sid Number(4),
  sname Varchar2(10),
  m1 Number(3)
);
```

STUDENT

sid	sname	m1
-----	-------	----

ORACLE => RDBMS => create & maintain the database

SQL
PL/SQL

SQL => 5 sub languages

DDL
DRL
DML
DCL
TCL

DDL:

CREATE
ALTER
DROP
TRUNCATE
RENAME
FLASHBACK
PURGE

CREATE:

```
CREATE TABLE <name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>
  .
.]
);
```

STUDENT1

sid	sname	M1
1001	Ramu	70
1002	Kiran	50

100

sid	Number(4)
sname	Varchar2(10)
M1	Number(3)

CREATE TABLE student1

```
(
  sid Number(4),
  sname Varchar2(10),
  m1 number(3)
);
```

Output: Table created.

To see table structure:

Syntax:

DESC <table_name>;

DESC student1;

SETS PAGES 100 LINES 100;

Output:

**sid Number(4)
sname Varchar2(10)
m1 number(3)**

To see all tables list created by user:

user_tables:

It maintains all tables information

WRITE AS IT IS.

SELECT table_name FROM user_tables;

Output:

Table_name

**T1
STUDENT
STUDENT1**

Inserting Records:

Syntax of INSERT command:

**INSERT INTO <table_name>[(<column_list>)]
VALUES(<value_list>);**

Inserting single record:

INSERT INTO student1 VALUES(1001,'Ramu',70);

Output: 1 row created.

INSERT INTO student1 VALUES(1002,'Kiran',50);

Output: 1 row created.

COMMIT; --save

Inserting multiple records using parameters:

"parameter" concept is used to read value at run time.

&sid => parameter

INSERT INTO student1 VALUES(&sid,'&sname',&m1);

**enter value for sid: 1003
enter value for sname: Sai
enter value for m1: 80**

/

**enter value for sid: 1004
enter value for sname: Sravan
enter value for m1: 66**

/

**enter value for sid: 1005
enter value for sname: Vijay
enter value for m1: 77**

To see table data:

SELECT * FROM student1;

Inserting limited column values:

sid	sname	m1
1006	Prasad	

```
INSERT INTO student1(sid,sname)
VALUES(1006,'Prasad');
```

Inserting limited column values by
changing order:

1007	Sravan	
------	--------	--

```
INSERT INTO student1(sname,sid)
VALUES('Sravan',1007);
```

Example-2:

Employee

empno	ename	job	sal	doj
5001	A	CLERK	5000	25-dec-2021
5002	B	MANAGER	8000	20-AUG-2019

empno	Number(4)	100000.00
ename	Varchar2(10)	Number(8,2)
job	Varchar2(10)	
sal	Number(8,2)	
doj	Date	

```
CREATE TABLE employee
(
  empno Number(4),
  ename Varchar2(10),
  job Varchar2(10),
  sal Number(8,2),
  DOJ Date
);
```

Output: Table created.

Inserting record:

```
INSERT INTO employee
VALUES(5001,'A','CLERK',6000,'25-DEC-2021');
```

```
INSERT INTO employee
VALUES(5002,'B','MANAGER',8000,'17-aug-2019');
```

```
insert into employee
values(&empno,&ename,&job,&sal,&doj);
Enter value for empno: 5003
Enter value for ename: C
Enter value for job: CLERK
Enter value for sal: 15000
Enter value for doj: 13-oct-2018
old 1: insert into employee
values(&empno,&ename,&job,&sal,&doj)
new 1: insert into employee
values(5003,'C','CLERK',15000,'13-oct-2018')
```

1 row created.

SQL>

SQL> /

Enter value for empno: 1004

Enter value for ename: D

Enter value for job: salesman

Enter value for sal: 4000

Enter value for doj: 16-sep-2021

old 1: insert into employee

values(&empno,&ename,&job,&sal,&doj)

new 1: insert into employee

values(1004,'D','salesman',4000,'16-sep-2021')

1 row created.

employee

empno	ename	job	sal	doj
5005	E		9000	

INSERT INTO employee(empno,ename,sal)
VALUES(5005,'E',9000);

Constraint:

- Constraint => restrict / limit / control

✓ Check (maths_marks >= 0 AND maths_marks <= 100) max marks: 100

maths_marks
78
59
625

0 TO 100

55 ✓

79 ✓

429 → ERROR

625 → ERROR

Gender

M	✓
F	✓
F	✓
M	✓
Z	→ ERROR → Invalid data

Check (Gender = 'M' OR Gender = 'F')

Constraint:

- Constraint => Restrict / Limit / Control
- Constraint is a rule that is applied on a column.
- Constraint restricts the user from entering invalid data.
- It is used to implement Data Integrity.
- Data Integrity means, maintaining accurate & quality data.

ORACLE SQL provides following Constraints:

- Primary Key
 - Not Null
 - Unique
 - Default
 - Check
 - References [Foreign Key]
- duplicate
null

Primary Key:

- should not accept duplicate values
- should not accept null values

Ex: *primary key*

<i>empno</i>	ename	job	sal
1001	Raju	Clerk	6000
1002	Kiran	Clerk	5000
1003	Raju	Manager	10000
1004	Sai	Clerk	5000
1001	Arun	Manager	15000
	Sravan	Manager	12000

duplicate
X
Null
X

Not Null:

- It does not accept null value.
- It accepts duplicate value.

Ex:

Employee

empno	ename	sal
1001	Ramu	5000
1002	Kiran	7000
1003	Ramu	6000
1004		10000

Not Null
Duplicate ✓
Null X
It demands for ename.

Unique:

- It does not accept duplicate values.
- It accepts null values.

Ex:

STUDENT

AdmnNo	Sname	Scity
1234	Ramu	Hyd
1235	Kiran	Mumbai
	Vijay	Pune
1234	Arun	Delhi

Unique
Null ✓
Duplicate X

Customer

cid	cname	gmail_id	mobile_num
1001	A	abc@gmail.com	90123 45678
1002	B		90123 45678
1003	C	qwe@gmail.com	90123 45678

Unique
Unique
Null ✓
Null ✓
Duplicate X

1003 C qwe@gmail.com 0123 45678
X duplicate

Constraint	Duplicate	Null
Primary Key	X	X
Not Null	✓	X
Unique	X	✓

PK = Not Null + Unique

Default:

- It is used to apply default value to a column.
- When for all records value is same then apply default value for that column.

STUDENT

sid	sname	college_name	college_city	Fee
1	Ramu	nareshit	hyd	15000
2	Sai	nareshit	hyd	15000
3	Vijay	nareshit	hyd	15000
				6000

default 'nareshit'
default 'hyd'
default 15000

Check:

- It is used to apply our own conditions on column.

sid	sname	m1
1	A	67
2	B	78
3	C	554

max marks: 100

0 TO 100

Check (m1) >= 0
AND
m1 <= 100
554 >= 0 = T
554 <= 100 = F
All T

References [Foreign Key]:

- Foreign Key refers to Primary Key values of another table.
- IN Foreign Key column, another table's Primary key values only accepted. Other than those values will not be accepted.

Ex:

COURSE

PK

cid	cname
10	Java
20	SQL Server
30	Python
40	C#.Net

STUDENT

sid	sname	cid
1	A	30
2	B	10
3	C	40
4	D	80

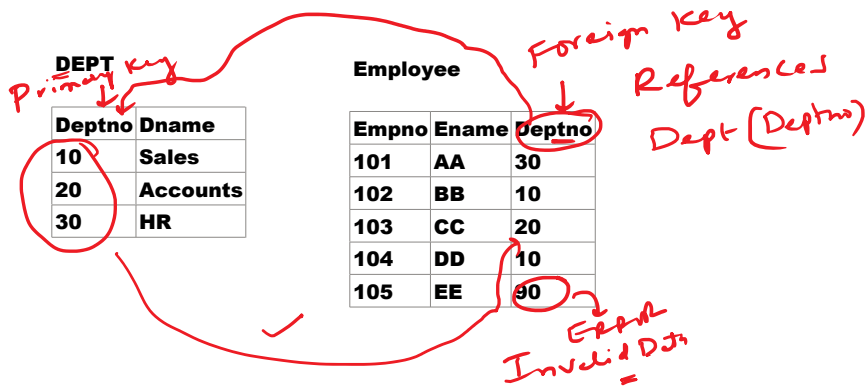
Foreign Key
References
Course (cid)

→ ERROR
Invalid data

DEPT

Employee

Foreign Key
References



Constraint => Rule applied on column

PK	not accept duplicate not accept null
Unique	not accept duplicate accepts null
Not Null	not accept null accepts duplicate
Check	to apply our own conditions
Default	to apply default value to column
References {FK}	Foreign Key refers to PK values of another table.

CREATE:

```

CREATE TABLE <name>
(
  <field_name> <data_type> [constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>,
  <field_name> <data_type> constraint <con_name> <con_type>
  .
]
);

```

Examples on Constraints:

Handwritten notes: "PK" points to sid; "Not Null" points to M1; "Check" points to M1; "Max marks: 100" and "0 to 100" are circled.

STUDENT2

sid	sname	M1
1001	A	67
1002	B	50
1001		
	C	88
1003		90
1004	A	50

Handwritten notes: "dup x" points to 1001; "null x" points to empty row; "duplication" points to 1004 A 50.

sid	Number(4)	PK
sname	Varchar2(10)	Not Null
M1	Number(3)	Check

```

CREATE TABLE student2
(
  sid Number(4) primary key,
  sname Varchar2(10) not null,

```

m1 Number(3) Check(m1>=0 AND m1<=100)
);
Output: Table created.

INSERT INTO student2 VALUES(1,'A',70);
Output: 1 row created.

INSERT INTO student2 VALUES(2,'B',50);
Output: 1 row created.

INSERT INTO student2 VALUES(1,'C',60);
Output: ERROR: unique constraint violated

INSERT INTO student2 VALUES(null,'C',60);
Output: Error: cannot insert null into student2.sid

INSERT INTO student2 VALUES(3,'A',65);
Output: 1 row created.

INSERT INTO student2 VALUES(4,null,60);
Output:
Error: cannot insert null into student2.sname

INSERT INTO student2 VALUES(5,'E',789);
Output: ERROR: check constraint violated.

Example-2:

STUDENT3

sid	sname	cname	ccity	Fee
1001	A	nareshit	Hyd	15000
1002	B	nareshit	Hyd	15000
1003	A	nareshit	Hyd	15000
1004	D	nareshit	Hyd	15000
1001	E	nareshit	Hyd	15000

sid	Number(4)	Unique
sname	Varchar2(10)	
cname	Varchar2(10)	DEFAULT
ccity	Varchar2(10)	Default
Fee	Number(7,2)	Default

CREATE TABLE student3
(
sid Number(4) Unique,
sname Varchar2(10),
cname Varchar2(10) Default 'nareshit',
ccity Varchar2(10) Default 'Hyd',
Fee Number(7,2) Default 15000
);
Output: Table created.

INSERT INTO student3 VALUES(1,'A');
Output: ERROR: not enough values

INSERT INTO student3(sid,sname) VALUES(1,'A');
Output: 1 row created.

INSERT INTO student3(sid,sname) VALUES(2,'B');
Output: 1 row created.

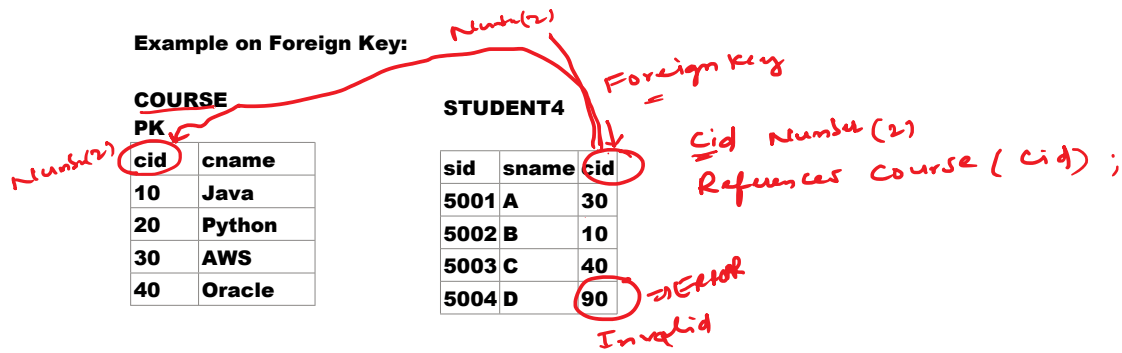
INSERT INTO student3(sid,sname) VALUES(1,'C');
Output: ERROR: unique constraint violated

INSERT INTO student3(sid,sname) VALUES(null,'D');
Output: 1 row created.

INSERT INTO student3(sid,sname) VALUES(5,'E');
Output: 1 row created.

SELECT * FROM student3;

Example on Foreign Key:



Course

cid	Number(2)	PK
cname	Varchar2(10)	

STUDENT4

sid	Number(4)	
sname	Varchar2(10)	
cid	Number(2)	Foreign Key Course cid

CREATE TABLE course

(
cid Number(2) Primary Key,
cname Varchar2(10)
);

Output: Table created.

CREATE TABLE student4

(
sid Number(4),
sname Varchar2(10),
cid Number(2) REFERENCES course(cid)
);

NOTE:

- To set foreign key, PK column and FK column data type & field size should be matched
- Column name need not to be same.

INSERT INTO course VALUES(10,'JAVA');
INSERT INTO course VALUES(20,'PYTHON');
INSERT INTO course VALUES(30,'AWS');
INSERT INTO course VALUES(40,'ORACLE');
COMMIT;

SELECT * FROM course;

INSERT INTO student4(1001,'A',30);
output: 1 row created.

INSERT INTO student4(1002,'B',10);
output: 1 row created.

INSERT INTO student4(1002,'B',90);
output: error: integrity constraint violated

Naming Constraints:

- We can give names to the constraints.
- "constraint" keyword is used to define name for

constraint.

- When we define a constraint, it's better to give constraint name. If we don't define constraint name ORACLE defines a name implicitly.
- ORACLE defined name will be prefixed with "sys_c"
- "sys_c" & 6 digit random number will be taken as constraint name.
- To disable or enable or drop the constraints name is required.

Creating Table with constraint types & names:

student11

sid	sname	m1
-----	-------	----

sid	Number(4)	PK	c1
sname	Varchar2(10)	Not Null	c2
m1	Number(3)	Check	c3

```
CREATE TABLE student11
(
  sid number(4) constraint c1 primary key,
  sname varchar2(10) constraint c2 not null,
  m1 number(3) constraint c3 check(m1>=0 and m1<=100)
);
```

Output: Table created.

user_constraints:

It maintains all constraints information.

DESC user_constraints;

```
SELECT constraint_name, constraint_type,
table_name FROM user_constraints;
```

```
SELECT constraint_name, constraint_type,
table_name FROM user_constraints
WHERE table_name='STUDENT11';
```

SQL:

Queries

5 sub languages

DDL:

CREATE

ALTER

DROP

TRUNCATE

RENAME

FLASHBACK

PURGE

ALTER:

- ALTER => Change

STUDENT

sid|sname|scity|GENDER

after some days => add a column GENDER

- Using ALTER command we can change structure of the table.

- Using ALTER command we can:

- add the columns → ADD
- rename the columns → RENAME COLUMN
- drop the columns → DROP
- modify the field sizes → MODIFY
- modify the data types → MODIFY
- add the constraints → ADD CONSTRAINT
- rename the constraints → RENAME
- disable constraints → DISABLE
- enable the constraints → ENABLE
- drop the constraints → DROP

Syntax of ALTER command:

```
ALTER TABLE <table_name> [ADD(<field_definitions>)]
                        [RENAME COLUMN <old_column_name>] TO <new_col_name>]
                        [DROP COLUMN <column_name>]
                        [DROP(<colun_list>)]
                        [MODIFY(<new_field_definitions>)]
                        [ADD CONSTRAINT <constraint_name> <con_type(<column>)]
                        [RENAME CONSTRAINT <old_name> TO <new_name>]
                        [DISABLE CONSTRAINT <constraint_name>]
                        [ENABLE CONSTRAINT <constraint_name>]
                        [DROP CONSTRAINT <constraint_name>;]
```

Example on ALTER command:

STUDENT

sid|sname

CREATE TABLE student

```
(
sid number(4),
sname varchar2(10)
);
```

Output: Table created.

Adding a column [m1 column]:

ALTER TABLE student ADD m1 number(3);

Output: Table Altered.

Desc student;

sid
sname
m1

Adding Multiple Columns [m2, m3 columns]:

**ALTER TABLE student ADD(m2 number(3),
m3 number(3));**

Changing column name [change m3 col name as maths]:

**ALTER TABLE student
RENAME COLUMN m3 TO maths;**

Dropping one column [maths column]:

```
ALTER TABLE student
DROP COLUMN maths;
```

--we can drop one column only using **DROP COLUMN**

Dropping multiple columns [m1 & m2]:

```
ALTER TABLE student
DROP(m1,m2);
```

Modifying data type [sid => number(4) => char(10)]:

```
ALTER TABLE student
MODIFY sid char(10);
```

Modifying the field size [sname => varchar2(10) => 10 to 20] :

```
ALTER TABLE student
MODIFY sname varchar2(20);
```

Can we decrease field size?

Yes. But, we can decrease up to max string length in column data.

sname → varchar2(20)
↓

Ramu — 4
Sravan — 6
Kiran — 5
naresh — 6
sai — 7
↓
varchar2(6)
↓
up to 6 only we can decrease
=

Adding Constraint:

[Add PK to sid]

Alter table student add primary key(sid);

```
ALTER TABLE student
ADD CONSTRAINT c5 Primary Key(sid);
```

Disabling Constraint:

```
ALTER TABLE student
DISABLE CONSTRAINT c5;
-- temporarily PK will not work
```

Enabling Constraint:

```
ALTER TABLE student
ENABLE CONSTRAINT c5;
```

Renaming Constraint:

```
ALTER TABLE student
RENAME CONSTRAINT c5 TO abc;
```

Dropping Constraint:

**ALTER TABLE student
DROP CONSTRAINT abc;**

DDL Commands:

**CREATE
ALTER
DROP
FLASHBACK
PURGE
TRUNCATE
RENAME**

**DROP
FLASHBACK
PURGE**

DROP:

- used to drop [delete] the table.
- When we drop the table, it goes to **RECYCLEBIN**.

Syntax:

DROP TABLE <table_name> [PURGE];

Ex:

DROP TABLE employee;

-- employee table will be placed in recyclebin

To see recyclebin [dropped tables list]:

show recyclebin

employee

FLASHBACK:

- used to restore the dropped table.

Syntax:

FLASHBACK TABLE <table_name> TO BEFORE DROP;

Ex:

FLASHBACK TABLE employee TO BEFORE DROP;

PURGE:

- is used to delete dropped table from recyclebin.

Syntax:

PURGE TABLE <table_name>;

Ex:

PURGE TABLE employee;

Dropping table permanent:

DROP TABLE employee;
PURGE TABLE employee;

OR

DROP TABLE employee PURGE;

Emptying recyclebin:

PURGE recyclebin;

DROP	drop the table. it goes to recyclebin
FLASHBACK	restore the dropped table
PURGE	to delete table from recyclebin

TRUNCATE:

- it is used to delete all records from the table.
- it deletes complete information from the table.

Syntax:

TRUNCATE TABLE <table_name>;

Ex:

TRUNCATE TABLE student;

- all records will be deleted.
- we cannot recollect truncated data.

student

=> TABLE

sid	sname
1	A
2	B
3	C

Structure

+

Data

TRUNCATE	it deletes table data. it will not delete table structure.
DROP	it deletes entire table. It means, table data + table structure will be deleted

RENAME:

- used to rename the database objects like tables.

Syntax:

RENAME <old_name> TO <new_name>;

Ex:

RENAME student3 TO std;

DDL

CREATE	to create the tables
---------------	----------------------

ALTER	to change structure of table
DROP	to drop the table
FLASHBACK	to restore from recyclebin
PURGE	to delete from recyclebin
TRUNCATE	to delete all records
RENAME	to change the table name

DRL / DQL:

- DRL => Data Retrieval Language
- DQL => Data Query Language
- Retrieval => opening existing data
- Query => is a request that is sent to the database
- DRL delas with data retrievals.

ORACLE SQL provides only one DRL command. i.e:
SELECT

SELECT:

- used to retrieve the data from database.

Syntax:

default

```

SELECT [DISTINCT/ALL] <column_list / *>
FROM <table_list>
[WHERE <condition>]
[GROUP BY <grouping_Column_list>]
[HAVING <group_Condition>]
[ORDER BY <column_list> ASC/DESC];

```

ENGLISH
SENTENCES
WORDS

SQL
QUERIES
CLAUSES

CLAUSE => part of the query

Using **SELECT** command we can retrieve:

- single record
- a set of records
- all records
- limited columns
- all columns

Displaying single record:

Display the emp record whose name is SMITH:

```

SELECT * FROM emp
WHERE ename='smith';

```

no rows selected

NOTE:

- SQL is not case sensitive. But, string comparison is case sensitive.

'smith' = 'SMITH' => FALSE
no rows selected

'SMITH' = 'SMITH' => TRUE
displays smith record

column list
Tell name

```

SELECT empno,ename,sal FROM emp
WHERE ename='SMITH';

```

```

SELECT empno,ename,sal FROM emp
WHERE ename='SMITH';

```

-- displays SMITH record

Handwritten notes: "Take name" points to ename, "Condition" points to WHERE.

SELECT	to specify the column list
FROM	to specify the table name
WHERE	to write the condition on rows

Execution Order:

```

FROM
WHERE
SELECT

```

Display the emp record whose empno is 7499:

```

SELECT * FROM emp
WHERE empno=7499;

```

Display a set of records:

Display all managers records:

```

SELECT empno,ename,job
FROM emp
WHERE job='MANAGER';

```

Display all records:

```

SELECT * FROM emp;

```

Display limited columns:

emp

empno	ename	job	sal	mgr	comm	hiredate	deptno
-------	-------	-----	-----	-----	------	----------	--------

ename	job	sal
-------	-----	-----

```

SELECT ename,job,sal FROM emp;

```

Display all columns:

```

SELECT * FROM emp WHERE deptno=10;

```

Operators in SQL:

- Operator:
- Operator is a symbol that is used to perform operations like arithmetic operations or logical operations.

Oracle SQL provides following operators:

Arithmetic	+ - * /				
Relational (or) Comparison	<	>	<=	>=	=
				equals	
					!= (or) <> (or) ^=
					not equals

IN C/Java:

```

+   -   *   /   5%2
5/2 = 2

```

IN SQL:

Arithmetic	+ - * /
Relational (or) Comparison	< > <= >= = != (or) <> (or) ^= equals not equals
Logical	AND OR NOT
SET	UNION UNION ALL INTERSECT MINUS
Special	IN NOT IN BETWEEN AND NOT BETWEEN AND IS NULL IS NOT NULL LIKE NOT LIKE
Concatenation	
Other	Any All Exists Not Exists PIVOT UNPOVOT

+ - * / 5%2
5/2 = 2

IN SQL:

+ - * /
5/2 = 2.5

mod() => function
mod(5,2) => remainder 1

Arithmetic Operators:
are used to perform arithmetic operations.

+	Addition
-	Subtraction
*	Multiplication
/	Division

Examples on Arithmetic Operators:

Calculate 100+200:

SELECT 100+200 FROM dual;

100+200

300

SELECT 100+200 as total FROM dual;

TOTAL

300

SELECT 100+200 total FROM dual;

TOTAL

300

SELECT 100+200 as "total" FROM dual;

total

300

SELECT 100+200 as total marks FROM dual;
ERROR:

SELECT 100+200 as total_marks FROM dual;

TOTAL_MARKS

300

SELECT 100+200 as "total marks" FROM dual;

total marks

300

Column Alias:

- Column Alias is another name or alternative name of a column.
- To give column alias we use "AS" keyword.
- Using "AS" keyword is optional.
- to maintain case or to give alias in multiple words specify alias name in double quotes.

Dual:

- "DUAL" is a built-in table [readymade table].
- DUAL table created in "sys" schema [user].
- It has 1 column & 1 row
- column name is: DUMMY Varchar2(1) => X [data].

SYS

DUAL
DUMMY
X

SELECT 100+200 FROM emp;

emp => 14 rows

300
300
300
.
.
300 => 14 times

SELECT 100+200 FROM dual;

300 => dual => 1row

- If our requirement is getting one value as the result then use DUAL.
- to work with non-database values, we can use DUAL.

Calculate annual salary:

emp

empno	ename	sal
1001	A	5000
1002	B	8000
1003	C	6000

**SELECT empno,ename,sal,
sal*12 as "Annual Salary"
FROM emp;**

Calculate TA, HRA, TAX & GROSS salaries:

employee

empno	ename	sal
1001	A	6000
1002	B	9000
1003	C	4000

TA => 10% on sal
HRA => 20% on sal
TAX => 5% on sal
GROSS => sal+TA+HRA-TAX

```

SELECT empno,ename,sal,
sal*0.1 as TA,
sal*0.2 as HRA,
sal*0.05 as TAX,
sal+sal*0.1+sal*0.2-sal*0.05 as GROSS
FROM emp;

```

Assignment:

sid	sname	M1	M2	M3
1	A	60	90	80
2	B	65	78	34

calculate total & average marks

Relational Operators / Comparison Operators:
are used to compare 2 values

ORACLE SQL provides following Relational Operators:

>	greater than
<	less than
>=	greater than or equals to
<=	less than or equals to
=	equals to
!= / <> / ^=	no equals to

Display the emp records whose salary is more than 2500:

```

SELECT empno,ename,sal
FROM emp
WHERE sal>2500;

```

Display the emp records whose salary is 3000 or more:

```

SELECT empno,ename,sal
FROM emp
WHERE sal>=3000;

```

Display the emp records whose salary is less than 2000:

```

SELECT empno,ename,sal
FROM emp
WHERE sal<2000;

```

Display th emp records whose salary is 1500 or less:

```

SELECT empno,ename,sal
FROM emp
WHERE sal<=1500;

```

Display the emp records who are working in 30

dept:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno=30;
```

Display all clerks records:

```
SELECT empno,ename,job
FROM emp
WHERE job='CLERK';
```

Display all emps records except clerks:

```
SELECT empno,ename,job
FROM emp
WHERE job!='CLERK';
```

Numbers:

1
2
3
4
5
.
.
10

Date

1-jan-2022

2-jan-2022

3-jan-2022

4-jan-2022

.

.

31-dec-2022

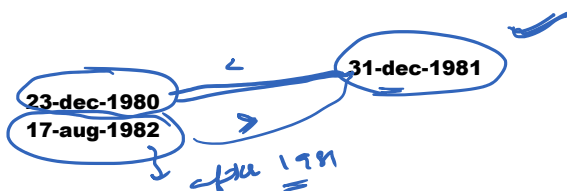
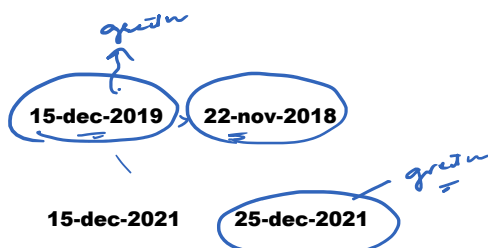
lowest
=

> 1-jan-2022

>

>

→ highest
date



Display the emp records who joined after 1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate>'31-dec-1981';
```

Display the emp records who joined before

1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate<'1-jan-1981';
```

LOGICAL OPERATORS:

are used to perform logical operations.

ORACLE SQL provides following Logical Operators:

- AND
- OR
- NOT

AND, OR operators are used to separate multiple conditions.

AND	All conditions should be satisfied
OR	At least one condition should be satisfied

Truth Table:

condition1	condition2	con1 AND con2	con1 OR con2
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

STUDENT

sid	sname	M1	M2	M3
1001	A	60	90	80
1002	B	50	30	70

Max marks: 100

Min marks: 40 for pass in each

Display the students records whose result is PASS:

```
SELECT * FROM student
WHERE m1>=40 AND m2>=40 AND m3>=40;
```

1001	A	60	90	80
------	---	----	----	----

Display the students records whose result is FAIL:

```
SELECT * FROM student
WHERE M1<40 OR M2<40 OR M3<40;
```

1002	B	50	30	70
------	---	----	----	----

Display all managers records whose salary is more than 2500:

```
SELECT empno,ename,job,sal
FROM emp
WHERE job='MANAGER' AND sal>2500;
```

Display all managers records whose salary is more than 2500 and hiredate must be after april 1981:

```
SELECT empno,ename,job,sal,hiredate
FROM emp
WHERE job='MANAGER' AND sal>2500
AND hiredate>'30-apr-1981';
```

Display all managers and clerks records:

```

SELECT empno,ename,job
FROM emp
WHERE job='MANAGER' OR job='CLERK';

```

AND	used to perform logical AND operations [ALL T]
OR	used to perform logical OR operations [Min 1 T]

NOT:
is used to perform logical NOT operations.

condn	Not(condn)
T	Not(T) => F
F	Not(F) => T

Display all managers records:

```

SELECT empno,ename,job FROM emp
WHERE job='MANAGER';

```

Display all emp records except managers:

```

SELECT empno,ename,job FROM emp
WHERE NOT(job='MANAGER');

```

Special Operators:
Special operators are also comparison operators.

IN:

Syntax:

```

<column_name> IN(<value_list>)

```

- used to compare a column value with a list of values.
- It avoids of writing multiple equality conditions using **OR**

Display the emp records whose empnos are 7499,7521,7900:

```

SELECT empno,ename,sal
FROM emp
WHERE empno=7499 OR empno=7521 OR empno=7900;

```

```

empno=7499 OR empno=7521 OR empno=7900

```

[OR]

```

empno IN(7499,7521,7900)

```

```

SELECT empno,ename,sal FROM emp
WHERE empno IN(7499,7521,7900);

```

Display the emp records whose names are:
SMITH, BLAKE AND WARD:

```
SELECT empno,ename,sal
FROM emp
WHERE ename IN('SMITH','BLAKE','WARD');
```

Display all managers and clerks records:

```
SELECT empno,ename,job
FROM emp
WHERE job IN('MANAGER','CLERK');
```

Display all emp records except clerks:

```
SELECT empno,ename,job
FROM emp
WHERE job NOT IN('CLERK');
```

Display all emp records who are working in
10 & 30 depts:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno IN(10,30);
```

Display all emp records who are not working
in deptno 30:

```
SELECT empno,ename,deptno
FROM emp
WHERE deptno NOT IN(30);
```

BETWEEN AND:

Syntax:

<column_name> BETWEEN <lower> AND <upper>

- It is used to compare column value with a range of values.

Examples:

Display the emp records whose salary is 2000 or greater and 3000 or less:

```
SELECT empno,ename,sal
FROM emp
WHERE sal BETWEEN 2000 AND 3000;
```

```
SELECT empno,ename,sal
FROM emp
WHERE sal BETWEEN 3000 AND 2000;
What si the output of query?
```

- a) **ERROR** b) displays records c) no rows selected
d) none of above

Display the emp records whose sal is <2000 or >3000:

```
SELECT empno,ename,sal
FROM emp
WHERE sal NOT BETWEEN 2000 AND 3000;
```

Display the emp records who joined in 1981:

who joined after 1981	hiredate>'31-dec-1981'
who joined before 1981	hiredate<'1-jan-1981'

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate BETWEEN '1-jan-1981' AND '31-dec-1981';
```

Display the emp records who are not joined in 1981:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate NOT BETWEEN '1-jan-1981' AND '31-dec-1981';
```

Display the emp records who joined in 1982,1983 & 1984:

```
SELECT empno,ename,hiredate
FROM emp
WHERE hiredate BETWEEN '1-jan-1982' AND '31-dec-1984';
```

LIKE:

Syntax:

<column_name> LIKE <text_pattern>

- used to compare column value with text pattern.

SQL provides 2 wildcard characters. They are:

%	replaces 0 or any no of chars
_	replaces 1 char

Display the emp records whose name is started with 'S':

```
SELECT empno,ename,sal
FROM emp
WHERE ename LIKE 'S%';
```

Display the emp records whose names are ended with S:

```
SELECT empno,ename,sal
FROM emp
WHERE ename LIKE '%S';
```

IN Windows OS,

to search for all jpg files	*.jpg
to search for all mp3 files	*.mp3
to search for all jpg files which are started with S	s*.jpg
to search for all jpg files in which 2nd char is a	?a*.jpg

Wildcard chars:

*	replaces 0 or any no of chars
?	replaces 1 char

Display the employee records whose name is started with 'A':

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE 'A%';
```

**Display the emp records whose name is ended with
"RD":**

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '%RD';
```

Display the emp records whose name is having 2nd char as A:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '_A%';
```

Display the emp records whose name's 3rd char is 'R':

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '__R%';
```

Display the emp records whose names are having 'A' char:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '%A%';
```

Display the emp records whose names are having 4 chars:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '____';
```

Display the emp records whose names are having 6 chars:

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename LIKE '_____';
```

Display the emp records who are getting 3 digit salary:

```
SELECT empno,ename,sal  
FROM emp  
WHERE sal LIKE '___';
```

**Display the emp records whose names are not started with
'S':**

```
SELECT empno,ename,sal  
FROM emp  
WHERE ename NOT LIKE 'S%';
```

NULL:

- **NULL means blank or empty**
- **When the value is unknown or unable to insert we give NULL.**

STUDENT

sid	sname	scity
1001	Ravi	Hyd
	Sai	Delhi

NULL

sid	sname	M1
1001	A	50
1002	B	0
1003	C	

NULL

space) ≠ NULL
0 ≠ NULL

NULL:

- NULL means blank or empty
- When the value is unknown or unable to insert we give NULL.
- Null is not equals to 0.
- Null is not equals to space.
- If NULL is participated in operation then definitely result will be NULL.

Ex:

100 + NULL = NULL

10 + 20 * null = NULL

- For Null comparison we cannot use = [equals to] operator. For Null comparison we must use "IS NULL" operator.

NULL value can be inserted in 2 ways:

- using NULL keyword
- by inserting limited column values

Ex:

CUSTOMER

cid	cname	ccity
-----	-------	-------

CREATE TABLE customer

```
(  
  cid number(4),  
  cname varchar2(10),  
  ccity varchar2(10)  
);
```

INSERT INTO customer VALUES(1,'A','Hyd');

1st way:

Inserting NULL using NULL keyword:

INSERT INTO customer VALUES(2,'B',null);

2nd way:

Inserting NULL by inserting limited column values:

INSERT INTO customer(cid,cname)
VALUES(3,'C');

IS NULL:

used to compare column value with NULL.

Syntax:

<column_name> IS NULL;

Display the emp records who are not getting commission:

```
SELECT empno,ename,sal,comm
FROM emp
WHERE comm IS null;
```

Display the emp records who are getting commission:

```
SELECT empno,ename,sal,comm
FROM emp
WHERE comm IS not null;
```

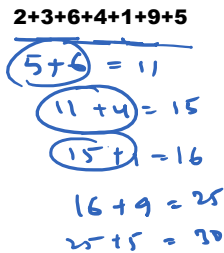
Concatenation Operator:
Symbol: ||

used to combine the 2 strings.

Ex:

```
SELECT 'Raj' || 'Kumar' FROM dual;
```

RajKumar



```
SELECT 'Raj' || ' ' || 'Kumar' FROM dual;
```

Raj Kumar

```
SELECT empno, ename FROM emp;
```

empno	ename
-----	-----

```
SELECT empno || ' ' || ename as "empno_name"
FROM emp;
```

empno_name

7369 SMITH
7499 ALLEN

SQL

DDL	create alter drop flashback purge truncate rename
DRL / DQL	SELECT
TCL	commit rollback

	savepoint
DML	Insert Update Delete Insert All Merge

TCL:

- TCL => Transaction Control Language
- It deals with transactions.
- Transaction is a series of actions [commands].

ORACLE SQL provides following TCL commands:

- COMMIT [Save]
- ROLLBACK [Undo All]

Ex:

withdraw Transaction

Reads card info => SELECT
select withdraw
enter amount => 5000 SELECT
success withdraw!
UPDATE the balance => UPDATE

5000
10000

Transaction must be successfully completed or aborted [cancelled].

COMMIT [Save]:

- It is used to save the transactions.
- When commit command is executed all the changes in ORACLE INSTANCE [RAM] will be applied to ORACLE DATABASE [HARD DISK].

Syntax:

COMMIT;

Ex:

COMMIT;
-- saves the transaction

NOTE:

ORACLE SERVER = Instance + Database
[RAM] [Hard Disk]

Rollback:

- is used to cancel the previous actions.
- it cancels uncommitted actions.
- After COMMIT, we cannot use ROLLBACK.

Syntax:

ROLLBACK [TO <save_point_name>];

Ex: ROLLBACK;

Example on ROLLBACK & COMMIT:

Test1

f1	f2
1	A
2	B

Note:

All DDL commands are auto-committed.

All DML commands are not auto-committed.

DDL command = DDL command + COMMIT

CREATE TABLE test1

```
(
f1 number(4),
f2 varchar2(10)
);
```

--committed

INSERT INTO test1 VALUES(1,'A');
INSERT INTO test1 VALUES(2,'B');

Select * FROM test1;

1	A
2	B

Rollback;

Select * FROM test1;

no rows selected

INSERT INTO test1 VALUES(1,'A');
INSERT INTO test1 VALUES(2,'B');

Select * FROM test1;

1	A
2	B

COMMIT;

ROLLBACK;

Select * FROM test1;

1	A
2	B

Case-1

CREATE => t1 => CREATE + COMMIT => committed

INSERT
INSERT
INSERT
INSERT

Rollback; — 4 actions cancelled

CREATE t1

INSERT

INSERT

CREATE t2

INSERT

committed
CREATE + COMMIT

```

INSERT
INSERT
CREATE t2 → CREATE + commit ✓
INSERT
INSERT
Rollback; → 2 actions cancelled ✓

```

SavePoint:

- used to set margin for Rollback.

Syntax:

Savepoint <save_point_name>;

Example on Savepoint:

CREATE Test1

```

SavePoint aaa;
INSERT INTO Test1 Values(11,'A');
INSERT INTO Test1 Values(12,'B');
SavePoint bbb;
INSERT INTO Test1 Values(13,'C');
INSERT INTO Test1 Values(14,'D');
SavePoint ccc;
INSERT INTO Test1 Values(15,'E');
INSERT INTO Test1 Values(16,'F');

```

Rollback TO aaa; -- 6 actions will be cancelled
 Rollback TO bbb; --4 actions will be cancelled
 Rollback TO ccc; -- 2 actions will be cancelled
 Rollback; -- 6 actions will be cancelled

TCL commands:

COMMIT [save]	used to save the transaction when commit is executed all changes of oracle instance will be applied to oracle database.
ROLLBACK	used to cancel previous actions it cancels uncommitted actions
SAVEPOINT	to set margin for rollback we use it

DML:

- DML => Data Manipulation Language
- Manipulation => Insert / Update / Delete
 - emp joined => INSERT
 - emp got promotion => UPDATE
 - emp resigned => DELETE
- It deals with the data.
- All DML commands are not auto-committed.
- After performing DML operation,
 - to save it use COMMIT.
 - to cancel it use ROLLBACK.

ORACLE SQL provides following DML commands:

- INSERT
- UPDATE

- DELETE
- INSERT ALL
- MERGE

UPDATE:

- it is used to modify the data.
- using this command we can update:
 - single value of single record
 - multiple values of single record
 - a set of records
 - all records
 - using parameters

Syntax:

```
UPDATE <table_name>
SET <field_name>=<value>[,<field_name>=<value>,
<field_name>=<value>, .....]
[WHERE <condition>];
```

Updating single value of single record:

```
UPDATE emp SET sal=sal+1000
WHERE empno=7369;
```

Updating multiple values of single record:

```
UPDATE emp SET job='MANAGER', sal=6000
WHERE empno=7369;
```

Updating a set of records:

Increase 10% on sal to all managers:

```
UPDATE emp SET sal=sal+sal*0.1
WHERE job='MANAGER';
```

Updating all records:

Increase 20% on sal to all emps:

```
UPDATE emp SET sal=sal+sal*0.2;
```

Updating records using parameters:

Increase => 10% on sal => empno 7499

Increase => 20% on sal => empno 7900

Increase => 15% on sal => empno 7934

```
UPDATE emp SET sal=sal+sal*&per/100
WHERE empno=&empno;
enter value for per:10
enter value for empno:7499
```

/

```
enter value for per: 20
enter value for empno: 7900
```

/

```
enter value for per: 15
enter value for empno: 7934
```

Ex-1:

$$\begin{aligned} \text{Total} &= m_1 + m_2 + m_3 \\ &= (m_1 + m_2 + m_3) / 3 \end{aligned}$$

Ex-1:

student

sid	sname	m1	m2	m3	total	avrg
1	A	60	80	80		
2	B	50	30	70		

$$\begin{aligned} \text{total} &= m_1 + m_2 + m_3 \\ \text{avrg} &= (m_1 + m_2 + m_3) / 3 \end{aligned}$$

CREATE TABLE student

```
(  
  sid number(4),  
  sname varchar2(10),  
  m1 number(3),  
  m2 number(3),  
  m3 number(3),  
  total number(3),  
  avrg number(5,2)  
);
```

```
INSERT INTO student(sid,sname,m1,m2,m3)  
VALUES(1,'A',60,80,70);
```

```
INSERT INTO student(sid,sname,m1,m2,m3)  
VALUES(2,'B',50,30,90);
```

COMMIT;

```
Select * FROM student;
```

```
UPDATE student SET total=m1+m2+m3,  
avrg=(m1+m2+m3)/3;
```

Assignment:
employee

empno	ename	job	sal	ta	hra	tax	gross
1	A	clerk	5000				

calculate ta, hra, tax & gross?

10% on sal as TA
20% on sal as HRA
5% on sal as TAX
GROSS = sal+TA+HTA-TAX

```
UPDATE employee SET ta=sal*0.1,  
hra=sal*0.1, tax=sal*0.05,  
gross=sal+sal*0.1+sal*0.2-sal*0.05;
```

Transfer all deptno 10 emps to deptno 20:

```
UPDATE emp SET deptno=20  
WHERE deptno=10;
```

Increase salaries & commissions as following:

If emps are getting commission,
increase 20% on sal as salary
increase 10% on comm as commission

```
UPDATE emp SET sal=sal+sal*0.2, comm=comm+comm*0.1  
WHERE comm is not null;
```

Increase 15% on sal to the emps who are having more than 40years experience:

**UPDATE emp SET sal=sal+Sal*0.15
WHERE (sysdate-hiredate)/365>40;**

DELETE:

- used to delete the records.
- using this command we can delete:
 - single record
 - a set of records
 - all records
 - using parameters

Syntax:

```
DELETE [FROM] <table_name>  
[WHERE <condition>];
```

Deleting single record:

Delete an emp record whose empno is 7934:

**DELETE FROM emp
WHERE empno=7934;**

Deleting a set of records:

Delete all clerks records:

DELETE FROM emp WHERE job='CLERK';

Deleting all records:

**DELETE FROM emp;
(or)
DELETE emp;**

Deleting records using parameters:

**DELETE FROM emp WHERE empno=&empno;
enter value for empno: 7900**

**/
enter value for empno: 7499**

**/
enter value for empno: 7934**

**Delete all emp records wh are having more than
40years experience:**

**DELETE FROM emp
WHERE (sysdate-hiredate)/365>40;**

Delete all emp records who joined in 1982;

**DELETE FROM emp
WHERE hiredate>='1-jan-1982' AND hiredate<='31-dec-1982';**

(or)

**DELETE FROM emp
WHERE hiredate BETWEEN '1-jan-1982' AND '31-dec-1982';**

(or)

```
DELETE FROM emp
WHERE hiredate LIKE '%82';
```

SQL

DDL	c a d t r f p
DRL	select
DML	insert update delete insert all, merge
DCL	grant, revoke
TCL	commit rollback savepoint

DCL / ACL:

- **DCL => Data Control Language.**
- **ACL => Accessing Control Language**
- It deals with data accessibility.
- Using DCL, we can implement table level security.

ORACLE SQL provides following DCL commands:

- **GRANT**
- **REVOKE**

GRANT:

- used to give permissions to other users on database objects like tables, views.

Syntax:

```
GRANT <privileges_list> ON <table_name>
TO <user_name>;
```

Revoke:

- Revoke command is used to cancel the permissions from other users

Syntax:

```
REVOKE <privileges_list> ON <table_name>
FROM <user_name>;
```

c##batch4pm:

```
GRANT select ON emp TO c##batch2pm;
```

c##batch2pm:

```
SELECT * FROM c##batch4pm.emp;
--displays the records
```

```
delete from emp;
--ERROR: insufficient privileges
```

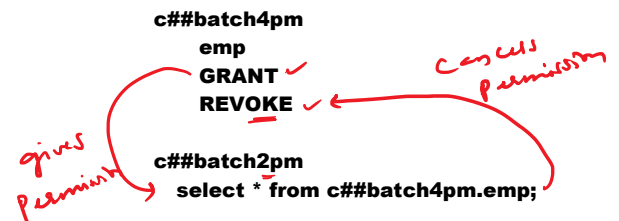
c##batch4pm:

```
GRANT insert,update ON emp TO c##batch2pm;
```

c##batch2pm:

```
UPDATE c##batch4pm.emp SET sal=sal+1000;
14 rows updated.
```

c##batch4pm:



GRANT all ON emp TO c##batch2pm;

**REVOKE insert,update,delete ON emp
FROM c##batch2pm;**

REVOKE all On emp FROM c##batch2pm;

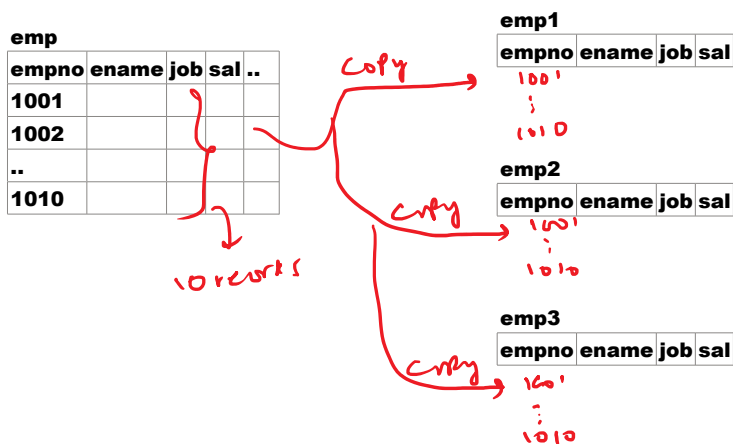
Insert All:

- introduced in Oracle 9i version.
- used to insert multiple records into multiple tables (or) single table.
- Normally, it is used to copy records from existing table & insert into multiple tables.
- It avoids of writing multiple INSERT commands.
- It can be used in 2 styles. They are:
 - Unconditional Insert All
 - Conditional Insert All

Unconditional Insert All:

Syntax:

```
INSERT ALL
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  INTO <table_name>[(<column_list>)] VALUES(<value_list>)
  .
  .
  <SELECT query>;
```



```
CREATE TABLE emp1
(
  empno number(4),
  ename varchar2(10),
  job varchar2(10),
  sal number(7,2)
);
```

```
CREATE TABLE emp2
(
  empno number(4),
  ename varchar2(10),
  job varchar2(10),
```

```

sal number(7,2)
);

CREATE TABLE emp3
(
  empno number(4),
  ename varchar2(10),
  job varchar2(10),
  sal number(7,2)
);

```

```

INSERT ALL
  INTO emp1 VALUES(empno,ename,job,sal)
  INTO emp2 VALUES(empno,ename,job,sal)
  INTO emp3 VALUES(empno,ename,job,sal)
SELECT empno,ename,job,sal FROM emp;

```

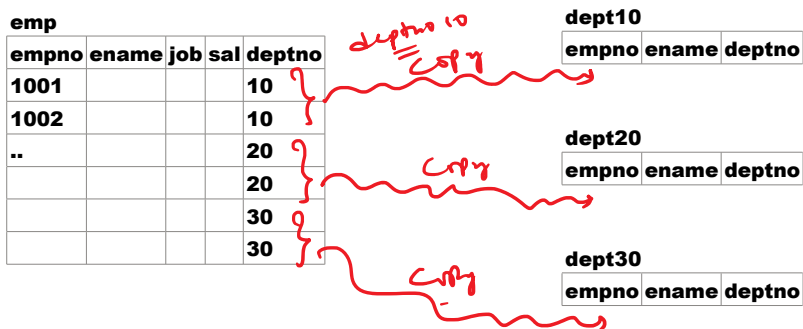
42 rows inserted.

Conditional Insert All:

```

INSERT ALL
  WHEN <condition-1> THEN
    INTO <table_name>(<column_list>) VALUES(value_list)
  WHEN <condition-2> THEN
    INTO <table_name>(<column_list>) VALUES(value_list)
  WHEN <condition-3> THEN
    INTO <table_name>(<column_list>) VALUES(value_list)
  .
  .
[ELSE
  INTO <table_name>(<column_list>) VALUES(value_list)]
<SELECT query>;

```



```

CREATE TABLE dept10
(
  empno number(4),
  ename varchar2(10),
  deptno number(2)
);

```

```

CREATE TABLE dept20
(
  empno number(4),
  ename varchar2(10),
  deptno number(2)
);

```

```

CREATE TABLE dept30
(

```

```
empno number(4),
ename varchar2(10),
deptno number(2)
);
```

```
INSERT ALL
WHEN deptno=10 THEN
  INTO dept10 VALUES(empno,ename,deptno)
WHEN deptno=20 THEN
  INTO dept20 VALUES(empno,ename,deptno)
WHEN deptno=30 THEN
  INTO dept30 VALUES(empno,ename,deptno)
SELECT empno,ename,deptno FROM emp;
```

MERGE command:

used to apply the changes of one table to its replica.

Syntax:

```
MERGE INTO <target_table_name> <alias>
USING <source_table_name> <alias>
ON(<condition>)
WHEN matched THEN
  UPDATE query
WHEN not matched THEN
  INSERT query;
```

ORACLE:

RDBMS => create & maintain the database in the form of tables.

SQL  **ORACLE DATABASE**
PL/SQL

SQL:

SQL => Query Language => Queries

5 Sub Languages

DDL	CREATE ALTER DROP TRUNCATE RENAME FLASHBACK PURGE
DRL	SELECT
DML	INSERT DELETE UPDATE INSERT ALL MERGE
DCL	GRANT REVOKE
TCL	COMMIT [SAVE] ROLLBACK SAVEPOINT

$s.cid = t.cid$

Branch

Customer1

cid	cname	ccity
1001	A	Bangalore
1002	B	Goa
1003	C	Mumbai
1004	D	Kolkata
1005	E	Pune

HQ

Customer2

cid	cname	ccity
1001	A	Hyd
1002	B	Delhi
1003	C	Mumbai

Syntax of Merge Command:

```
MERGE INTO <target_table_name> <alias>
USING <source_table_name> <alias>
ON(<condirion>)
WHEN matched THEN
  <Update query>
WHEN not matched THEN
  <INSERT query> ;
```

Example on MERGE:

CREATE TABLE customer1

```
(
cid number(4),
cname varchar2(10),
ccity varchar2(10)
);
```

```
INSERT INTO customer1 VALUES(1,'A','Hyd');
INSERT INTO customer1 VALUES(2,'B','Delhi');
INSERT INTO customer1 VALUES(3,'C','Mumbai');
COMMIT;
```

Copying Table:

```
CREATE TABLE customer2
AS
SELECT * FROM customer1;
```

--with select query result a new table customer2 will be created.

--exact copy of customer1 will be created

```
UPDATE customer1 SET ccity='Bangalore'
WHERE cid=1;
```

```
UPDATE customer1 SET ccity='Goa'
WHERE cid=2;
```

```
INSERT INTO customer1 VALUES(4,'D','Pune');
INSERT INTO customer1 VALUES(5,'E','Chennai');
COMMIT;
```

MERGE:

```
MERGE INTO customer2 t  
USING customer1 s  
ON(s.cid = t.cid)  
WHEN matched THEN  
    UPDATE SET t.cname=s.cname, t.ccity=s.ccity;  
WHEN not matched THEN  
    INSERT VALUES(s.cid,s.cname,s.ccity);
```

Built-In Functions

Saturday, July 23, 2022 5:14 PM

SQL => 4GL [4th Generation Language]

It provides readymade commands & readymade functions

Readymade functions are called "Built-In Functions" or "Predefined Functions".

Built-In Functions:

- **Oracle developers already define some functions & placed in Oracle Database. These functions are called "Built-In Functions / Predefined Functions".**
- **Function => Task / Action / Job**
- **Every function performs particular task.**
- **To make our tasks easier ORACLE DEVELOPERS defined "Built-In Functions".**

SQL built-functions can be categorized as following:

- **String Functions [Text Functions / Char Functions]**
- **Aggregate Functions [Group functions]**
- **Math Functions [Number Functions]**
- **Conversion Functions**
- **Date Functions**
- **Miscellaneous Functions**

String Functions [Text Functions / Char Functions]:

lower()

Ltrim()

Instr()

Soundex()

upper()

Rtrim()

Substr()

ASCII()

lower()
upper()
initcap()
length()
concat()

Ltrim()
Rtrim()
Trim()

Lpad()
Rpad()

Instr()
Substr()

Replace()
Translate()
Reverse()

Soundex()
ASCII()
Chr()